

Datos que persisten

Las clases ya funcionan, pero todo se borra al cerrar el programa. Aquí los objetos empiezan a guardarse en una base de datos.

Rubén Schnettler • Docente

INACAP Valparaíso • Analista Programador

BLOQUE 4

RECUPERACIÓN

Con qué se llega hasta aquí

✓ Lo cubierto hasta aquí

- Clases, objetos, atributos y métodos
- Constructor, herencia y encapsulamiento
- Diagrama de clases con sus relaciones
- El caso escrito en Python y ejecutando



LO QUE TODAVÍA NO FUNCIONA

Cada vez que el programa termina, **los objetos desaparecen**. Al volver a ejecutarlo hay que crearlos de nuevo a mano. Un sistema que olvida todo entre una ejecución y la siguiente no le sirve a nadie.

Eso es lo que se resuelve en este bloque: que el dato sobreviva al programa que lo creó.

ACTIVACIÓN • A DÓNDE VAMOS

Lo que hay que construir al terminar

El producto • 50 % • grupal

- Un programa en Python **CRUD** conectadas a una base de datos con operaciones
- **Autenticación** de usuario para entrar al sistema
- **Consumo de APIs** externas: clima e indicadores económicos
- Almacenamiento de los datos obtenidos
- Uso **crítico e iterativo** de herramientas de inteligencia artificial

La defensa • 50 % • individual

Cada integrante explica **su aporte** y fundamenta las decisiones tomadas en:

- La implementación del CRUD y la conexión a la base
- La autenticación y el consumo de APIs
- El uso que se le dio a la IA durante el desarrollo

Es la **Unidad 2**, sobre el caso EcoTech Solutions, y pesa un **30 %**. Grupos de dos o tres. **Se entrega solo el código**, sin informe.

ACTIVACIÓN • EL VOCABULARIO

CRUD: las cuatro cosas que un programa hace con un dato

Es una sigla en inglés que nombra las cuatro operaciones básicas sobre datos almacenados. Cualquier sistema que administre información las implementa, sin excepción.

C**Create**

Crear un registro nuevo

Contratar a un empleado y
darlo de alta en el sistema

R**Read**

Consultar lo guardado

Listar los empleados del
área de desarrollo

U**Update**

Modificar un registro

Cambiar el correo de una
persona que lo actualizó

D**Delete**

Eliminar un registro

Dar de baja a quien ya no
trabaja en la empresa

Hasta ahora esas cuatro operaciones se hicieron **sobre una lista en memoria**, y desaparecían al cerrar el programa. En esta unidad se harán **sobre una base de datos**, y sobrevivirán.

BLOQUE

01

El dato que sobrevive

Qué significa persistir, y por qué una base de datos y no un archivo

EL PUNTO DE PARTIDA

Todo se pierde al cerrar

```
class Empleado:
    def __init__(self, nombre, correo):
        self.nombre = nombre
        self.correo = correo

empleados = []
empleados.append(Empleado("Ana Rojas", "ana@ecotech.cl"))
print(len(empleados)) # 1
```

Se ejecuta, imprime 1, y termina.

Terminal

```
$ python main.py
1
$ python main.py
1
```



SIEMPRE 1, NUNCA 2

La lista vive en la memoria del proceso. Cuando el proceso termina, **la memoria se libera** y el objeto deja de existir.

CONCEPTO

Persistir: escribir en un medio que no se borra

PERSISTENCIA

Capacidad de un objeto de **existir más allá del proceso que lo creó**: sus datos se escriben en un medio que sobrevive.

MEDIO	CÓMO SE VE	PARA QUÉ SIRVE	LÍMITE
Archivo de texto	Un .txt con líneas	Registrar lo que pasó, sin volver a leerlo	Buscar un dato obliga a recorrerlo entero
Archivo JSON	Un .json con estructura	Guardar configuración o pocos datos	Se carga completo en memoria
Base de datos	Un archivo que se consulta con SQL	Guardar, buscar, filtrar y relacionar	Hay que aprender otro lenguaje

Aquí se usa una **base de datos**: el caso pide buscar, filtrar y relacionar. Un archivo sirve para escribir; una base sirve para *preguntar*.

LA DIFERENCIA QUE IMPORTA

Guardar no es lo mismo que consultar

Con un archivo

```
# ¿Cuántos empleados del área  
# de Desarrollo hay?  
total = 0  
for linea in open("empleados.txt"):  
    partes = linea.split(";")  
    if partes[2] == "Desarrollo":  
        total += 1
```

Hay que recorrer el archivo completo y programar la búsqueda.

Con una base de datos

```
# La misma pregunta  
cursor.execute(  
    "SELECT COUNT(*) FROM empleado "  
    "WHERE area = ?", ("Desarrollo",)  
)
```



LA PREGUNTA SE DECLARA

No se programa *cómo* buscar: se declara **qué** se quiere. El motor de la base resuelve el resto.

EJERCICIO 1 • EN VOZ ALTA

5 minutos

¿Dónde guardaría cada cosa?

Para cada dato, indique si conviene un **archivo** o una **base de datos**.

1 Los 400 empleados de la empresa, que se consultan a diario

2 El registro de cada vez que alguien inició sesión

3 La dirección del servidor y el puerto

4 El nombre de la empresa y el color del logotipo

5 Las horas trabajadas por cada empleado, mes a mes

6 Los proyectos, con sus empleados asignados

**PISTA**

La pregunta que decide es: **¿voy a tener que buscar, filtrar o cruzar este dato con otro?** Si la respuesta es sí, es base de datos.

RESPUESTA · EJERCICIO 1 Revisión en plenario

Archivo o base de datos

N.º	DATO	DÓNDE	POR QUÉ
1	Los 400 empleados	Base	Se consulta, se filtra y se relaciona con otras entidades
2	El registro de inicios de sesión	Archivo	Solo se escribe y se lee cuando hay un incidente. Es una bitácora
3	Servidor y puerto	Archivo	Es configuración: se lee al arrancar y no cambia
4	Nombre y color del logotipo	Archivo	También configuración
5	Las horas por empleado y mes	Base	Se suman, se agrupan y se cruzan con el empleado
6	Los proyectos con sus empleados	Base	Es una relación entre dos entidades: exactamente para lo que existe una base

Configuración y bitácoras van a archivo. **Toda entidad del negocio va a la base de datos:** es lo mismo que salió del diagrama de clases.

BLOQUE

02

SQLite y el módulo sqlite3

La base de datos que ya viene instalada con
Python

LA HERRAMIENTA

SQLite: una base de datos en un archivo

Sin servidor No hay que instalar ni arrancar nada. La base **es un archivo** en la carpeta del proyecto

Sin instalación El módulo `sqlite3` **viene con Python**. No se instala con `pip`

SQL de verdad Las mismas instrucciones que en MySQL, PostgreSQL o SQL Server. Lo que se aprende aquí se traslada

```
import sqlite3
print(sqlite3.sqlite_version)
```

```
$ python -c "import sqlite3; print(sqlite3.sqlite_version)"
3.45.1
```

>_

SE COMPRUEBA AHORA

Si esa línea imprime un número de versión, no falta nada por instalar.

EL VOCABULARIO MÍNIMO

Cinco instrucciones bastan para trabajar

connect() Abre la base. Si el archivo no existe, **lo crea**

execute() Ejecuta **una** instrucción SQL

close() Cierra la conexión y libera el archivo

cursor() El objeto que ejecuta las instrucciones y trae los resultados

commit() Confirma los cambios. **Sin esto no se guarda nada**

fetchall() Trae **todas** las filas del último SELECT, como lista



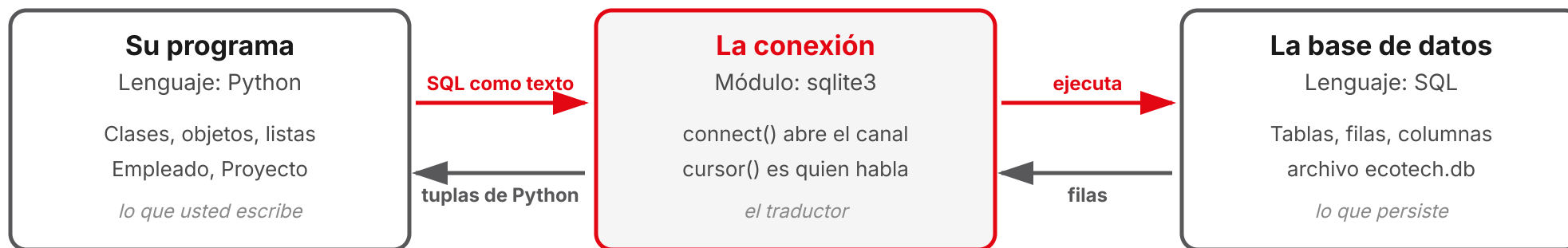
LA OMISIÓN MÁS COSTOSA

Un `execute()` que inserta datos y no va seguido de `commit()` **no guarda nada**. El programa no da error: simplemente, al volver a abrir la base, el dato no está.

EL PUNTO QUE MÁS CUESTA

Son dos lenguajes distintos en el mismo archivo

Python no sabe consultar una base de datos. La base de datos no entiende Python. Se comunican porque Python **envía texto escrito en SQL** y recibe de vuelta datos que sí entiende.



De ahí que el SQL vaya siempre **entre comillas** dentro del código Python: para Python es una cadena de texto cualquiera. Quien la interpreta es la base de datos, al otro lado de la conexión.

LA MISMA IDEA, EN CÓDIGO

Dónde termina Python y dónde empieza SQL

```
# Python: abre el canal
conexion = sqlite3.connect("ecotech.db")
cursor = conexion.cursor()

# Lo de adentro NO es Python: es SQL
cursor.execute("SELECT nombre FROM empleado")

# Python otra vez: recibe y recorre
filas = cursor.fetchall()
for fila in filas:
    print(fila[0])
```

execute() Entrega la instrucción SQL a la base. Python no la lee ni la comprende: solo la transporta

"SELECT ..." Para Python es una cadena de texto. Para la base es una orden. Por eso un error de SQL no lo detecta el editor

fetchall() Trae el resultado convertido a estructuras de Python: una lista de tuplas

fila[0] Cada fila es una tupla. Se accede por posición, en el orden en que se pidieron las columnas

Consecuencia práctica: **el editor no avisa si el SQL está mal escrito**. El error aparece recién al ejecutar, y lo lanza la base de datos, no Python.

VOCABULARIO DE PYTHON

Cuatro maneras de guardar varios datos juntos

La diferencia está en **si se pueden modificar** y en **cómo se accede a cada elemento**. Una consulta a la base devuelve unas y no otras.

ESTRUCTURA	SE ESCRIBE	SE ACCEDE	¿SE MODIFICA?	CUÁNDO SE USA
Lista	[1, 2, 3]	lista[0]	Sí	Una colección que crece o cambia: los empleados cargados en memoria
Tupla	(1, 2, 3)	tupla[0]	No	Un conjunto fijo de valores que van juntos: una fila que devuelve la base de datos
Diccionario	{"id": 1}	dic["id"]	Sí	Datos con nombre en lugar de posición: la respuesta de una API
Conjunto	{1, 2, 3}	No tiene orden	Sí	Valores sin repetir: los correos distintos de una lista con duplicados

El paréntesis identifica la estructura: [] lista • () tupla • { } con dos puntos, diccionario • { } sin dos puntos, conjunto.

POR QUÉ IMPORTA AQUÍ

La base de datos devuelve tuplas, no objetos

```
cursor.execute("SELECT id, nombre FROM empleado")
filas = cursor.fetchall()
```

```
# filas es una LISTA de TUPLAS
# [(1, 'Ana'), (2, 'Luis')]
```

```
for fila in filas:
    print(fila[0], fila[1])
```

```
# Y para volver a objetos:
empleados = [Empleado(f[1]) for f in filas]
```

fetchall() Devuelve una **lista**: se puede recorrer y contar

fila Cada elemento es una **tupla**: no se puede modificar

fila[0] Se accede por posición, en el orden en que se pidieron las columnas

[... for ...] Construye una lista nueva recorriendo otra. Aquí, una lista de objetos Empleado

Intentar `fila[0] = 99` falla: **una tupla no se modifica**. Es una protección, no una limitación: lo que vino de la base no debe alterarse por accidente.

PASO A PASO

La tabla se escribe por partes, no de una vez

Nadie escribe un **CREATE TABLE** completo de memoria. Se construye en cuatro decisiones, en este orden.

01 Qué se guarda · el nombre de la tabla

Una por cada clase del diagrama, en minúsculas y en singular.

```
CREATE TABLE empleado ( ... )
```

02 Qué datos tiene · una columna por atributo

Cada atributo del UML es una columna.

```
numero,
correo,
area
```

03 De qué tipo es cada uno

TEXT texto · **INTEGER** enteros · **REAL** decimales.

```
nombre TEXT,  
correo TEXT,  
area TEXT
```

04 Qué reglas debe cumplir · las restricciones

Qué no puede quedar vacío ni repetirse, y qué identifica la fila.

```
id      INTEGER PRIMARY KEY,
nombre TEXT NOT NULL,
correo TEXT NOT NULL UNIQUE
```

LAS TRES REGLAS QUE SE USAN SIEMPRE

Qué hace cada restricción, y qué pasa si falta

RESTRICCIÓN	QUÉ OBLIGA	QUÉ OCURRE SI NO ESTÁ
PRIMARY KEY	Identifica de forma única cada fila. En SQLite, si es INTEGER, se numera sola	No hay forma segura de referirse a una fila concreta para actualizarla o borrarla
NOT NULL	La columna no admite quedar vacía	Se pueden insertar empleados sin nombre, y el programa falla al mostrarlos
UNIQUE	El valor no puede repetirse en dos filas	Dos empleados con el mismo correo, y el inicio de sesión deja de ser confiable



POR QUÉ IMPORTA PARA LA EVALUACIÓN

Las restricciones son **validación que ocurre en la base de datos**, no en el programa: aunque alguien evite las validaciones de Python, la base rechaza el dato igual.

SE ESCRIBE EN PANTALLA

Crear la base y la tabla, línea por línea

crear_base.py

```
A import sqlite3

B conexion = sqlite3.connect("ecotech.db")
C cursor = conexion.cursor()

D cursor.execute("""
    CREATE TABLE IF NOT EXISTS empleado (
        id      INTEGER PRIMARY KEY,
        nombre  TEXT      NOT NULL,
        correo  TEXT      NOT NULL UNIQUE,
        area    TEXT
    )
""")

E conexion.commit()
F conexion.close()
```

- A Módulo de la biblioteca estándar: no se instala.
- B Abre `ecotech.db`. Si no existe, lo crea vacío.
- C El cursor es quien ejecuta. La conexión sola no ejecuta nada.
- D **IF NOT EXISTS**: crea la tabla solo si no está. Sin eso, la segunda ejecución falla.
- E Confirma. Aquí es cuando el archivo cambia de verdad.
- F Cierra. Deja el archivo libre para otro programa.

SE EJECUTA EN PANTALLA

Qué aparece después de ejecutar

Terminal

```
$ python crear_base.py
$ ls
crear_base.py  ecotech.db
```

El programa no imprime nada, y está bien: su efecto es el archivo nuevo.

```
proyecto/
src/
    crear_base.py
    ecotech.db ← la base
    .venv/
```

**ESE ARCHIVO ES LA BASE DE DATOS**

Se puede copiar, respaldar o borrar como cualquier archivo. Si se borra, se pierden los datos: por eso **nunca se sube al repositorio**.

CADA PALABRA DEL CREATE TABLE

Qué dice cada línea de la tabla

INTEGER Número entero. Para identificadores, cantidades y conteos

REAL Número con decimales. Para montos, horas y porcentajes

NOT NULL Obliga a que la columna traiga valor. Sin esto se cuelan filas a medias

TEXT Texto de cualquier largo. Para nombres, correos y descripciones

PRIMARY KEY Identifica la fila. Con INTEGER, SQLite lo numera solo

UNIQUE Prohíbe repetir el valor. Dos empleados no pueden tener el mismo correo

**ESTO TAMBIÉN ES SEGURIDAD**

Las restricciones no son decoración: son la **última defensa** del dato. Aunque el programa tenga un error y mande un correo repetido, la base lo rechaza.

EJERCICIO 2 • INDIVIDUAL

8 minutos

Escribir el CREATE TABLE de Proyecto

Proyecto

- id: int
- codigo: str
- nombre: str
- horasPresupuestadas: float
- activo: bool



Escriba la instrucción

- Una columna por atributo
- El tipo que corresponda a cada uno
- La clave primaria
- Las restricciones que hagan falta



DOS AVISOS

SQLite **no tiene tipo booleano**: se usa INTEGER con 0 y 1.
Y el código de proyecto no debería poder repetirse.

RESPUESTA · EJERCICIO 2

Revisión en plenario

La tabla proyecto, resuelta

```
CREATE TABLE IF NOT EXISTS proyecto (  
  id                INTEGER PRIMARY KEY,  
  codigo            TEXT    NOT NULL UNIQUE,  
  nombre            TEXT    NOT NULL,  
  horas_presupuestadas REAL  NOT NULL,  
  activo            INTEGER NOT NULL DEFAULT 1  
);
```

- 1 `horasPresupuestadas` pasa a `horas_presupuestadas`: en SQL también se usa `snake_case`.
- 2 Lleva decimales, así que es `REAL`, no `INTEGER`.
- 3 `activo` es `INTEGER` con 0 o 1, y arranca en 1 gracias al `DEFAULT`.
- 4 El código es `UNIQUE`: dos proyectos no pueden compartirlo.

BLOQUE

03

Del diagrama a las tablas

El modelo de clases ya dice cómo tiene que ser la base

LA TRADUCCIÓN

Cada parte del diagrama tiene su equivalente

EN EL DIAGRAMA DE CLASES	EN LA BASE DE DATOS	DETALLE
Una clase	Una tabla	En singular y en minúscula: empleado
Un atributo	Una columna	Con el tipo SQL equivalente
El identificador	PRIMARY KEY	Si el diagrama no lo trae, se agrega igual
Asociación o agregación	Una FOREIGN KEY	Va en el lado «muchos» de la relación
Composición	FOREIGN KEY con cascada	Si muere el todo, mueren las partes
Generalización	Una tabla con columna de tipo	O una tabla por hija

La base **se deduce del modelo que ya está hecho**. Si al pasar a tablas aparece algo que el diagrama no tenía, el diagrama estaba incompleto.

CONCEPTO

La clave foránea es la relación, escrita

```
CREATE TABLE registro_tiempo (  
  id          INTEGER PRIMARY KEY,  
  fecha       TEXT    NOT NULL,  
  horas       REAL    NOT NULL,  
  empleado_id INTEGER NOT NULL,  
  FOREIGN KEY (empleado_id)  
    REFERENCES empleado(id)  
    ON DELETE CASCADE  
)
```

- 1 **empleado_id** guarda el **id del empleado dueño** del registro. Esa columna es la relación.
- 2 **FOREIGN KEY ... REFERENCES** le dice a la base que ese número tiene que existir en **empleado**.
- 3 **ON DELETE CASCADE** es la composición del diagrama: al borrar el empleado, sus registros se borran con él.
- 4 La columna va en el lado **«muchos»**: un empleado tiene muchos registros, así que el id viaja al registro, no al revés.

EL CASO COMPLETO

Las tres tablas de EcoTech

empleado

id • PK
nombre
correo • UNIQUE
area
salario

registro_tiempo

id • PK
fecha
horas
empleado_id • FK

proyecto

id • PK
codigo • UNIQUE
nombre
horas_presupuestadas
activo



LO QUE FALTA Y HAY QUE DECIDIR

El diagrama dice que un empleado participa en varios proyectos y un proyecto tiene varios empleados. Una relación de **muchos a muchos** no cabe en una columna: necesita **una tabla propia**, con los dos identificadores.

EL CASO QUE SIEMPRE APARECE

Muchos a muchos: una tabla intermedia

```
CREATE TABLE asignacion (  
    empleado_id INTEGER NOT NULL,  
    proyecto_id INTEGER NOT NULL,  
    horas REAL,  
    PRIMARY KEY (empleado_id, proyecto_id),  
    FOREIGN KEY (empleado_id) REFERENCES empleado(id),  
    FOREIGN KEY (proyecto_id) REFERENCES proyecto(id)  
);
```

- 1 La tabla no representa una cosa del mundo: representa el **vínculo** entre dos cosas.
- 2 La clave primaria son **las dos columnas juntas**: el mismo empleado no puede estar dos veces en el mismo proyecto.
- 3 Puede llevar datos propios del vínculo, como las horas asignadas. Ese es el momento en que la tabla deja de ser solo un puente.
- 4 En el diagrama de clases esto aparece como una **clase asociación**.

EJERCICIO 3 • EN PAREJA 10 minutos

De estas dos clases a dos tablas

Departamento

- id: int
- nombre: str
- presupuesto: float

1 ◇ — 0..*

Empleado

- id: int
- nombre: str
- correo: str



Escriba las dos tablas

- Los CREATE TABLE completos
- Los tipos de cada columna
- Dónde va la clave foránea, y por qué ahí
- ¿Lleva o no lleva ON DELETE CASCADE?



LA PREGUNTA QUE DECIDE EL CASCADE

Es una **agregación**, rombo vacío: si se cierra el departamento, ¿desaparecen los empleados?

RESPUESTA · EJERCICIO 3

Revisión en plenario

Las dos tablas, resueltas

```
CREATE TABLE departamento (  
    id            INTEGER PRIMARY KEY,  
    nombre        TEXT      NOT NULL UNIQUE,  
    presupuesto   REAL      NOT NULL  
);  
  
CREATE TABLE empleado (  
    id            INTEGER PRIMARY KEY,  
    nombre        TEXT      NOT NULL,  
    correo        TEXT      NOT NULL UNIQUE,  
    departamento_id INTEGER,  
    FOREIGN KEY (departamento_id)  
        REFERENCES departamento(id)  
);
```

- 1 La clave foránea va en **empleado**, que es el lado «muchos» de la relación.
- 2 Va **sin NOT NULL**: la multiplicidad **0..*** admite que un empleado no tenga departamento asignado todavía.
- 3 **No lleva CASCADE**. Es agregación: si se cierra el departamento, los empleados siguen existiendo y quedan sin asignar.

BLOQUE

04

Las cuatro operaciones

Crear, leer, actualizar y borrar: eso es todo lo que
hace un sistema

CONCEPTO

CRUD: las cuatro cosas que se le hacen a un dato

LETRA	OPERACIÓN	EN SQL	EN EL SISTEMA
C	Create · crear	INSERT	Registrar un empleado nuevo
R	Read · leer	SELECT	Listar los empleados, buscar uno
U	Update · actualizar	UPDATE	Corregir un correo, cambiar de área
D	Delete · borrar	DELETE	Dar de baja a alguien que se fue

**POR QUÉ IMPORTA EL NOMBRE**

Casi cualquier sistema de gestión —el del taller, el del gimnasio, el de EcoTech— es **un CRUD sobre unas cuantas tablas**, más las reglas del negocio. Dominar estas cuatro operaciones es dominar el 80 % del trabajo.

C • CREAR

INSERT, línea por línea

```
A conexion = sqlite3.connect("ecotech.db")
B cursor = conexion.cursor()

C cursor.execute(
    "INSERT INTO empleado (nombre, correo, area) "
    "VALUES (?, ?, ?)",
D    ("Ana Rojas", "ana@ecotech.cl", "Desarrollo")
)

E conexion.commit()
F print("Id asignado:", cursor.lastrowid)
conexion.close()
```

- A Se abre la base que ya existe.
- B El cursor ejecuta.
- C La instrucción nombra las columnas y deja **un signo de interrogación por cada valor**. No se escribe el dato aquí.
- D Los valores van aparte, en una tupla, **en el mismo orden** que los signos.
- E Confirma. Sin esto la fila no queda.
- F `lastrowid` devuelve el id que la base acaba de asignar.

LO MÁS IMPORTANTE DEL BLOQUE

Por qué los valores van con signo de interrogación

Como NO se escribe

```
nombre = input("Nombre: ")
cursor.execute(
    f"INSERT INTO empleado (nombre) "
    f"VALUES ('{nombre}')"
)
```



INYECCIÓN SQL

El texto que escribe el usuario **se convierte en instrucción**. Si escribe `x'); DROP TABLE empleado; --`, la base ejecuta eso.

Como sí se escribe

```
nombre = input("Nombre: ")
cursor.execute(
    "INSERT INTO empleado (nombre) "
    "VALUES (?)",
    (nombre,)
)
```



CONSULTA PARAMETRIZADA

El signo de interrogación viaja como **dato**, nunca como instrucción. Lo que el usuario escriba se guarda tal cual, sin ejecutarse.

EL DETALLE QUE SE OMITE CON FRECUENCIA

La coma de la tupla de un solo valor

```
# Mal: no es una tupla
cursor.execute(sql, (nombre))

# Bien: la coma la convierte en tupla
cursor.execute(sql, (nombre,))

# También válido
cursor.execute(sql, [nombre])
```



EL ERROR QUE PRODUCE

ValueError: parameters are of unsupported type. Sin la coma, (nombre) es solo un texto entre paréntesis, y Python lo recorre letra por letra.

(x,) Tupla de un elemento. **La coma es obligatoria**

(x) No es tupla: es x entre paréntesis

R • LEER

SELECT: traer filas y recorrerlas

```
A cursor.execute("SELECT id, nombre, area "  
                  "FROM empleado "  
                  "WHERE area = ? "  
                  "ORDER BY nombre", ("Desarrollo",))  
  
B filas = cursor.fetchall()  
  
C for fila in filas:  
D     print(fila[0], fila[1], fila[2])
```

```
1 Ana Rojas Desarrollo  
4 Luis Vera Desarrollo
```

- A **SELECT** nombra las columnas que se quieren, **WHERE** filtra y **ORDER BY** ordena.
- B **fetchall()** trae **todas** las filas como una lista de tuplas.
- C Se recorre como cualquier lista de Python.
- D Cada fila es una tupla: se accede por posición, en el mismo orden del **SELECT**.
- 5 Para una sola fila se usa **fetchone()**, que devuelve la tupla o **None**.

U Y D

Actualizar y borrar, con su advertencia

Actualizar

```
cursor.execute(
    "UPDATE empleado "
    "SET area = ? "
    "WHERE id = ?",
    ("Operaciones", 7)
)
conexion.commit()
print(cursor.rowcount, "fila(s) modificada(s)")
```

Borrar

```
cursor.execute(
    "DELETE FROM empleado "
    "WHERE id = ?",
    (7,)
)
conexion.commit()
```

**SIN WHERE SE BORRA TODO**

Un DELETE FROM empleado sin WHERE vacía la tabla completa, sin preguntar y sin vuelta atrás.

LA COMPROBACIÓN QUE SE OLVIDA

rowcount: saber si el UPDATE hizo algo

```
cursor.execute("UPDATE empleado SET area = ? "
               "WHERE id = ?", (area, id_buscado))
conexion.commit()

if cursor.rowcount == 0:
    print("No existe un empleado con ese id")
else:
    print("Actualizado")
```

**POR QUÉ IMPORTA**

Un UPDATE que no encuentra la fila **no da error**: simplemente no modifica nada. Sin revisar rowcount, el programa le informa al usuario que guardó algo que nunca guardó.

rowcount Cuántas filas afectó la última instrucción

lastrowid El id que la base asignó en el último INSERT

EJERCICIO 4 · INDIVIDUAL

15 minutos

Las cuatro operaciones, en orden



ENUNCIADO

Sobre la tabla **proyecto** que escribió en el ejercicio 2, programe en un solo archivo: **1)** insertar tres proyectos · **2)** listarlos ordenados por nombre · **3)** cambiarle las horas a uno · **4)** desactivar otro poniendo `activo = 0`.

Todo con signos de interrogación. Al final, un **SELECT** que muestre cómo quedó la tabla.



Qué entregar

- El archivo que ejecuta sin error
- La salida en la terminal
- El `rowcount` de cada cambio



LOS TRES ERRORES MÁS FRECUENTES

Olvidar el `commit()` · concatenar en vez de parametrizar · el `DELETE` sin `WHERE`.

RESPUESTA · EJERCICIO 4

Revisión en plenario

Una solución posible

```
import sqlite3

conexion = sqlite3.connect("ecotech.db")
cursor = conexion.cursor()
datos = [("P-01", "Portal", 120.0),
         ("P-02", "App", 240.0),
         ("P-03", "Migración", 80.0)]
cursor.executemany("INSERT INTO proyecto (codigo, nombre, "
                  "horas_presupuestadas) VALUES (?, ?, ?)", datos)
cursor.execute("UPDATE proyecto SET activo = 0 WHERE codigo = ?",
              ("P-03",))
conexion.commit()
for fila in cursor.execute("SELECT * FROM proyecto"):
    print(fila)
conexion.close()
```

- 1 `executemany()` repite la instrucción con cada tupla.
- 2 Se desactiva con `UPDATE`, no con `DELETE`.

BLOQUE

05

Separar la clase del SQL

Dónde va el código que habla con la base, y por qué no va en la clase

EL PROBLEMA

La clase Empleado no tiene por qué saber SQL

Baja cohesión

```
class Empleado:
    def __init__(self, nombre, correo):
        self.nombre = nombre
        self.correo = correo

    def guardar(self):
        conexion = sqlite3.connect("ecotech.db")
        # ... SQL adentro de la clase ...
```



DOS TEMAS EN UNA CLASE

Sabe qué es un empleado **y** cómo se conecta a SQLite.

Alta cohesión

```
class Empleado:
    # solo lo que es un empleado
    ...

class EmpleadoRepositorio:
    # solo cómo se guarda y se lee
    ...
```



CADA CLASE, UN TEMA

La clase representa la cosa; el **repositorio** la lleva y la trae de la base. Es el criterio de cohesión, otra vez.

37

SE ESCRIBE EN PANTALLA

EmpleadoRepositorio • abrir y guardar

src/empleado_repositorio.py

```
import sqlite3

class EmpleadoRepositorio:
    A def __init__(self, ruta="ecotech.db"):
    B     self.__ruta = ruta

    C def guardar(self, empleado):
    D     with sqlite3.connect(self.__ruta) as con:
    E         cur = con.execute(
            "INSERT INTO empleado (nombre, correo, area) "
            "VALUES (?, ?, ?)",
            (empleado.nombre, empleado.correo, empleado.area))
    F         empleado.id = cur.lastrowid
    return empleado
```

- A La ruta del archivo llega por parámetro, con un valor por omisión. Así el mismo repositorio sirve para las pruebas con otra base.
- B Privada: nadie de afuera cambia la ruta a mitad de camino.
- C Recibe un **objeto** `Empleado`, no cinco parámetros sueltos.
- D `with` abre la conexión y la **confirma y cierra sola** al terminar.
- E La conexión también puede ejecutar directo, sin pedir cursor aparte.
- F Se le devuelve al objeto el id que asignó la base.

SE ESCRIBE EN PANTALLA

EmpleadoRepositorio • leer, actualizar y borrar

```
def obtener_todos(self):
    with sqlite3.connect(self.__ruta) as con:
        filas = con.execute(
            "SELECT id, nombre, correo, area "
            "FROM empleado ORDER BY nombre").fetchall()
    return [self.__a_objeto(f) for f in filas]

def actualizar_area(self, id_emp, area):
    with sqlite3.connect(self.__ruta) as con:
        cur = con.execute("UPDATE empleado SET area = ? "
                          "WHERE id = ?", (area, id_emp))

    return cur.rowcount

def __a_objeto(self, fila):
    emp = Empleado(fila[1], fila[2], fila[3])
    emp.id = fila[0]
    return emp
```

- 1 `obtener_todos` devuelve **objetos**, no tuplas. Quien lo llame trabaja con `empleado.nombre`, no con `fila[1]`.
- 2 Eso lo hace `__a_objeto`: convierte una fila en un objeto. Es privado porque nadie más lo necesita.
- 3 `actualizar_area` devuelve `rowcount`: quien llama puede saber si encontró la fila.
- 4 Toda la clase **habla de empleados**, y el SQL no se escapa de aquí.

SE ESCRIBE EN PANTALLA

El programa principal queda limpio

main.py

```
from empleado import Empleado
from empleado_repositorio import EmpleadoRepositorio

repo = EmpleadoRepositorio()

repo.guardar(Empleado("Ana Rojas", "ana@ecotech.cl", "Desarrollo"))
repo.guardar(Empleado("Luis Vera", "luis@ecotech.cl", "Operaciones"))

for emp in repo.obtener_todos():
    print(emp.id, emp.nombre, ".", emp.area)
```

Segunda ejecución

```
1 Ana Rojas · Desarrollo
2 Luis Vera · Operaciones
3 Ana Rojas · Desarrollo
4 Luis Vera · Operaciones
```



LA SEGUNDA EJECUCIÓN LO CONFIRMA

La segunda ejecución **encuentra lo de la primera.** Ese es el cambio que trajo este bloque.

EJERCICIO 5 • EN PAREJA 8 minutos

¿Qué le falta a este repositorio?

```
class ProyectoRepositorio:
    def __init__(self):
        self.con = sqlite3.connect("ecotech.db")

    def guardar(self, codigo, nombre, horas):
        sql = f"INSERT INTO proyecto (codigo, nombre, "
            f"horas_presupuestadas) VALUES " \
            f"('{codigo}', '{nombre}', {horas})"
        self.con.execute(sql)

    def listar(self):
        return self.con.execute("SELECT * FROM proyecto")
```

🔍 Encuentre cuatro problemas

- Uno es de seguridad, y es grave
- Uno hace que los datos no se guarden
- Uno deja un recurso abierto
- Uno rompe la cohesión que se acaba de explicar

RESPUESTA • EJERCICIO 5

Revisión en plenario

Los cuatro problemas

N.º	PROBLEMA	POR QUÉ IMPORTA	CÓMO SE CORRIGE
1	SQL concatenado	Inyección SQL: el texto del usuario se ejecuta	Parámetros con signo de interrogación
2	Falta el commit	El INSERT no queda guardado	Usar with, que confirma solo
3	La conexión no se cierra	Queda abierta y bloquea el archivo	Abrir y cerrar en cada método
4	Recibe datos sueltos	Termina conociendo el orden de los campos	Recibir un objeto Proyecto



LOS CUATRO APARECEN JUNTOS, Y NO ES CASUALIDAD

Es el aspecto del código generado sin revisión: funciona la primera vez y falla en producción.

BLOQUE

06

Cuando algo falla

Excepciones, validación y mensajes que sirven
para algo

EL PUNTO DE PARTIDA

Cuando la operación falla a medio camino

QUÉ PASA	QUÉ LEVANTA PYTHON	QUÉ VE EL USUARIO SI NO SE MANEJA
El correo ya existe en la base	<code>sqlite3.IntegrityError</code>	Una pantalla de error con la traza completa
La tabla no existe todavía	<code>sqlite3.OperationalError</code>	Lo mismo, y el programa se interrumpe
El archivo de la base está bloqueado	<code>sqlite3.OperationalError</code>	El sistema se congela sin explicación
Escribió letras donde iba un número	<code>ValueError</code>	El programa termina a mitad de camino
El id buscado no existe	<code>nada: devuelve None</code>	Un error más adelante, lejos de la causa



LA ÚLTIMA FILA ES LA MÁS GRAVE

Un error que **no levanta excepción** sigue circulando como `None` hasta reventar lejos de su causa.

LA ESTRUCTURA

try, except y finally, línea por línea

```
A try:
B     repo.guardar(empleado)
      print("Empleado registrado")

C except sqlite3.IntegrityError:
      print("Ese correo ya está registrado")

D except sqlite3.Error as error:
      print("Error de base de datos:", error)

E finally:
      print("Operación terminada")
```

- A **try** encierra lo que **puede fallar**. Solo eso, no el programa entero.
- B Si todo sale bien, se ejecuta y los **except** se saltan.
- C El primero es **el caso concreto** que se sabe que puede ocurrir, con un mensaje que el usuario entiende.
- D El segundo es la red de seguridad: cualquier otro error de la base. Va **después** del específico.
- E **finally** se ejecuta siempre, con error o sin él. Sirve para cerrar lo que haya quedado abierto.

EL ORDEN IMPORTA

Del error más concreto al más general

Al revés: el específico nunca se ejecuta

```
try:
    ...
except sqlite3.Error:
    print("Error de base de datos")
except sqlite3.IntegrityError:
    print("Correo repetido") # inalcanzable
```



POR QUÉ

IntegrityError **es un** sqlite3.Error: el primer except ya lo atrapó. Es la herencia de la unidad anterior, aplicada a los errores.

Nunca esto

```
try:
    ...
except:
    pass
```



EL EXCEPT MUDO

Atrapa **cualquier** error y no hace nada con él. El programa sigue como si todo hubiera salido bien, con la base a medio escribir y nadie enterado. En una revisión de código esto se rechaza sin discusión.

ANTES DE LA BASE

Validar la entrada es más barato que atrapar el error

```
def validar_empleado(nombre, correo, horas):  
    if not nombre or len(nombre.strip()) < 3:  
        raise ValueError("El nombre es obligatorio")  
  
    if "@" not in correo:  
        raise ValueError("El correo no es válido")  
  
    if horas < 0 or horas > 12:  
        raise ValueError("Las horas van entre 0 y 12")
```

raise Levanta una excepción a propósito, con el mensaje que se quiera

ValueError La excepción estándar para «el dato no sirve»

.strip() Quita los espacios de los extremos. Un nombre de puros espacios no es un nombre



LAS TRES VALIDACIONES EXIGIBLES

Obligatorio —que venga—, **formato** —que tenga la forma correcta— y **rango** —que esté dentro de lo posible—.

DETALLE TÉCNICO

Un mensaje de error que sirve y uno que no

MENSAJE	¿SIRVE?	POR QUÉ
Error	No	No dice qué pasó, ni dónde, ni qué hacer
<code>sqlite3.IntegrityError: UNIQUE constraint failed</code>	No	Es el mensaje del motor, no del sistema. El usuario no sabe qué es una restricción
Ese correo ya está registrado. Use otro o busque al empleado existente	Sí	Dice qué pasó y qué hacer a continuación
No se pudo guardar. Revise el archivo de registro	A medias	Sirve si el registro existe y alguien lo revisa



LA REGLA

El usuario recibe **qué pasó y qué hacer**. El detalle técnico va al registro del sistema, donde lo lee quien programa. Mostrar la traza completa en pantalla, además, **le entrega información al atacante**.

EJERCICIO 6 • INDIVIDUAL

12 minutos

Poner las defensas donde faltan

```
def registrar_horas(repo, id_empleado, texto_horas):  
    horas = float(texto_horas)  
    repo.guardar_registro(id_empleado, horas)  
    print("Horas registradas")
```



Corrija la función

- Valide las horas antes de guardar
- Atrape el caso de texto no numérico
- Atrape el error de base de datos
- Deje un mensaje útil en cada caso



PISTA

Hay **tres** cosas que pueden fallar aquí, y solo una de ellas es de la base de datos.

RESPUESTA · EJERCICIO 6

Revisión en plenario

La función con sus defensas

```
def registrar_horas(repo, id_empleado, texto_horas):  
    try:  
        horas = float(texto_horas)  
    except ValueError:  
        print("Las horas deben ser un número. Ejemplo: 7.5")  
        return  
  
    if horas ≤ 0 or horas > 12:  
        print("Las horas de una jornada van entre 0 y 12")  
        return  
  
    try:  
        repo.guardar_registro(id_empleado, horas)  
        print("Horas registradas")  
    except sqlite3.IntegrityError:  
        print("Ese empleado no existe")  
    except sqlite3.Error as e:  
        print("No se pudo guardar:", e)
```

- 1 El **try** encierra **una sola línea**: la que puede fallar.
- 2 La validación de rango no necesita excepción: un **if** basta.
- 3 El **IntegrityError** significa que la clave foránea no encontró al empleado.

BLOQUE

07

La IA escribiendo código de datos

Qué entrega bien, qué entrega mal, y cómo se
revisa

EL CRITERIO DE LA ASIGNATURA

Lo que la rúbrica pide, en una frase

CRITERIO 2.1.5

«Aplica criterios técnicos en la validación del código generado por herramientas de IA, **justificando su adopción, modificación o descarte.**»

DECISIÓN	CUÁNDO	QUÉ HAY QUE DOCUMENTAR
Adoptar	Cumple con los criterios técnicos	Qué se revisó y qué se comprobó
Modificar	La idea sirve, la implementación no	Qué se cambió y por qué
Descartar	No cumple o resuelve otro problema	Por qué no servía y qué se hizo

Pegar lo que devolvió la herramienta sin análisis **se considera insuficiente**, y es fácil de detectar: el código generado tiene marcas reconocibles.

LA HERRAMIENTA

El asistente vive dentro del editor

No hace falta salir a una página web y volver pegando código. **GitHub Copilot** se integra en Visual Studio Code y trabaja sobre el archivo abierto, de modo que conoce las clases que usted ya escribió. Es gratuito para estudiantes con la cuenta institucional.

Sugerencia en línea

Escribe la firma de un método y propone el cuerpo, en gris.

Tab acepta · Esc descarta

Chat lateral

Se le pregunta en lenguaje natural sobre el proyecto abierto.

Ctrl + Alt + I

Edición sobre la selección

Se marca un fragmento y se pide una modificación concreta sobre él.

Ctrl + I

La ventaja es también el riesgo: **como ve su código, sus propuestas parecen correctas.** Sugiere lo que encaja con lo escrito, no necesariamente lo que resuelve el problema.

CÓMO SE LE PIDE

Del comentario vago a la instrucción dirigida

PEDIDO VAGO

```
# funcion para guardar empleado
```

Devuelve una función genérica: SQL concatenado y sin manejo de errores.

PEDIDO DIRIGIDO

```
# Metodo de EmpleadoRepositorio que inserta un  
# Empleado en la tabla empleado de ecotech.db.  
# Usar parametros con ? , no f-strings.  
# Envolver en try/except sqlite3.IntegrityError  
# y devolver el id generado.
```

Las cinco letras de CLARO, en el editor

Contexto Nombre de la clase, de la tabla y del archivo de base de datos

Labor El verbo exacto: insertar, consultar, actualizar

Alcance Solo el método, sin la clase completa

RoI Dentro del repositorio, no en el programa principal

Orden Las restricciones técnicas: parámetros, manejo de errores, valor de retorno

EL HÁBITO QUE SE EVALÚA

Pedir, leer, corregir y volver a pedir

La guía exige un uso **crítico e iterativo**: la primera respuesta casi nunca es la definitiva.

01 Pedir con el contexto puesto

Comentario dirigido, con la tabla y las restricciones técnicas declaradas.

02 Leer antes de aceptar

Si hay una línea que no sabe explicar, todavía no es suyo. Preguntarle al chat qué hace esa línea es parte del proceso.

03 Corregir sobre la propuesta

«Reemplaza la concatenación por parámetros» · «Ese except es demasiado amplio».

04 Registrar la decisión

Qué se adoptó, qué se modificó y qué se descartó. Esa tabla es evidencia del criterio 2.1.5.

En la defensa se pregunta por el paso 3: **qué le corrigió a la herramienta y por qué.**

QUÉ REVISAR SIEMPRE

Las seis marcas del código generado sin revisar

1 **SQL concatenado** con f-strings en lugar de parámetros

2 **Sin manejo de errores**, o con `except: pass`

3 **SELECT *** en vez de nombrar las columnas

4 **Sin cerrar** la conexión, o sin `with`

5 **Credenciales escritas** dentro del código

6 **La lógica de negocio metida en el SQL**, fuera de las clases



POR QUÉ OCURRE

La herramienta optimiza que el ejemplo **funcione en la primera prueba**, no que resista en producción. Todas estas marcas producen código que corre bien la primera vez.

EJERCICIO 7 • EN PAREJA 12 minutos

Revisar un fragmento generado

```
# Generado por una herramienta de IA
import sqlite3

def buscar_empleado(nombre):
    conn = sqlite3.connect("ecotech.db")
    c = conn.cursor()
    c.execute(f"SELECT * FROM empleado WHERE nombre = '{nombre}'")
    resultado = c.fetchall()
    return resultado
```

 Complete la tabla del informe

- Qué aspecto se revisó
- Qué problema tiene
- Qué decisión se toma
- Con qué fundamento técnico

**SE EVALÚA LA JUSTIFICACIÓN**

Decir «está malo» no sirve. Hay que decir **qué criterio incumple y qué se hizo.**

RESPUESTA · EJERCICIO 7 Revisión en plenario

La tabla que va en el informe

ASPECTO	PROBLEMA DETECTADO	DECISIÓN	FUNDAMENTO
Construcción de la consulta	El nombre se concatena	Modificar	Permite inyección. Se parametriza
Selección de columnas	SELECT * trae el salario	Modificar	Se nombran las columnas necesarias
Manejo de errores	No hay try	Modificar	El error de la base cae al usuario
Cierre de la conexión	Nunca se cierra	Modificar	Se usa with
Tipo de retorno	Devuelve tuplas crudas	Modificar	Se devuelven objetos Empleado

Se conserva **la idea** —buscar por nombre— y se reescribe la implementación. Esa distinción es lo que el criterio pide justificar.

BLOQUE

08

La evaluación de esta parte

Qué se entrega, con qué se mide y qué se defiende

EL ENCARGO

Un programa en Python con base de datos



LA SITUACIÓN DE EVALUACIÓN

En grupos, un programa con **operaciones CRUD conectadas a una base de datos**, más autenticación y consumo de servicios externos, usando de forma crítica herramientas de IA.

Los servicios externos y la autenticación van en el bloque siguiente.



Cómo se pondera

- Producto grupal • **50 %**
- Defensa individual • **50 %**
- Cada uno defiende su aporte



LA MITAD SE EVALÚA EN LA EXPOSICIÓN

Se pregunta por qué se tomó cada decisión: el diseño de las tablas, el manejo de errores y qué se hizo con lo que sugirió la IA.

EL INSTRUMENTO

Los criterios que se evalúan en el producto

#	CRITERIO	DÓNDE SE DEMUESTRA
2.1.1	Implementa clases y métodos desde el diagrama UML validado	Las clases coinciden con el diagrama
2.1.2	Aplica principios de POO, sin automatización	Encapsulamiento y cohesión visibles
2.1.3	Emplea librerías oficiales de base de datos	sqlite3, sin dependencias raras
2.1.4	Maneja errores y excepciones	try y validación de entradas
2.1.5	Valida el código generado por IA	La tabla de análisis, con fundamento

El criterio 2.1.2 dice **«sin automatización del código»**: el programa debe ser coherente con el modelo, no un archivo generado que casualmente funciona.

CÓMO SE ORGANIZA

Cómo organizar el proyecto

```
ecotech/  
  src/  
    empleado.py  
    registro_tiempo.py  
    empleado_repositorio.py  
    base.py          ← crea las tablas  
    main.py  
  diagramas/        ← el UML de la ES1  
  .gitignore
```

La guía exige solo el **código fuente comprimido**; esta organización es una propuesta.



LO QUE NO SE SUBE AL REPOSITORIO

El archivo `ecotech.db` y la carpeta `.venv/`. La base cambia en cada ejecución: subirla genera conflictos en cada mezcla.

```
# .gitignore  
*.db  
.venv/  
__pycache__/  
.env
```

ACTIVIDAD • EN GRUPO 60 minutos

El CRUD completo del caso

**ENUNCIADO**

Tomen **dos clases relacionadas** de su diagrama y llévenlas a base de datos: las tablas, el repositorio de cada una y un `main.py` que ejecute las cuatro operaciones y muestre el resultado.

**Y DESPUÉS, CON INTELIGENCIA ARTIFICIAL**

Cuando esté **escrito y ejecutando**, pídanle a una herramienta que lo revise. Anoten qué sugirió, qué adoptaron, qué modificaron y qué descartaron, con su fundamento.

**Qué entregar**

- El script que crea las tablas
- Los dos repositorios
- El `main.py` con el CRUD
- La salida en la terminal
- La tabla de análisis de la IA

CIERRE DEL BLOQUE

Con qué se sale de esta parte

✓ Lo que queda listo

- La base de datos creada desde el diagrama
- Las cuatro operaciones funcionando
- El repositorio, con el SQL separado de las clases
- Errores atrapados y entradas validadas

→ Lo que viene

- Pedirle datos a un servicio externo
- Autenticación, y dónde no van las claves
- Validar lo que llega de afuera
- El proyecto integrador

✓ EL CAMBIO DE ESTA PARTE, EN UNA FRASE

El sistema dejó de olvidar. **Lo que se registra hoy sigue ahí mañana**, y se puede consultar, corregir y dar de baja.

DATOS QUE PERSISTEN

Un objeto sin base de datos dura lo que dura el programa

Las clases modelan el mundo; la base lo
recuerda.

TI3V21 • Programación Orientada a Objeto Seguro • INACAP Valparaíso